GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training

Jiezhong Qiu
qiujz16@mails.tsinghua.edu.cn
Tsinghua University

Qibin Chen
cqb19@mails.tsinghua.edu.cn
Tsinghua University

Yuxiao Dong
yuxdong@microsoft.com
Microsoft Research, Redmond

Jing Zhang
zhang-jing@ruc.edu.cn
Remin University

Hongxia Yang
yang.yhx@alibaba-inc.com
DAMO Academy, Alibaba Group

Ming Ding
dm18@mails.tsinghua.edu.cn
Tsinghua University

Kuansan Wang
kuansan.wang@microsoft.com
Microsoft Research, Redmond

Jie Tang*
jietang@tsinghua.edu.cn
Tsinghua University

ABSTRACT

Graph representation learning has emerged as a powerful technique for addressing real-world problems. Various downstream graph learning tasks have benefited from its recent developments, such as node classification, similarity search, and graph classification. However, prior arts on graph representation learning focus on domain specific problems and train a dedicated model for each graph dataset, which is usually non-transferable to out-of-domain data. Inspired by the recent advances in pre-training from natural language processing and computer vision, we design Graph Contrastive Coding (GCC)[1]—a self-supervised graph neural network pre-training framework—to capture the universal network topological properties across multiple networks. We design GCC's pre-training task as subgraph instance discrimination in and across networks and leverage contrastive learning to empower graph neural networks to learn the intrinsic and transferable structural representations. We conduct extensive experiments on three graph learning tasks and ten graph datasets. The results show that GCC pre-trained on a collection of diverse datasets can achieve competitive or better performance to its task-specific and trained-from-scratch counterparts. This suggests that the *pre-training* and *fine-tuning* paradigm presents great potential for graph representation learning.

CCS CONCEPTS

• **Information systems** → **Data mining**; **Social networks**; • **Computing methodologies** → **Unsupervised learning**; **Neural networks**; **Learning latent representations**.

KEYWORDS

graph representation learning; graph neural networks; graph pre-training; self-supervised learning; contrastive learning

ACM Reference Format:
Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '20), August 23–27, 2020, Virtual Event, CA, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3394486.3403168

1 INTRODUCTION

Hypothesis. *Representative graph structural patterns are universal and transferable across networks.*

Over the past two decades, the main focus of network science research has been on discovering and abstracting the universal structural properties underlying different networks. For example, Barabasi and Albert show that several types of networks, e.g., World Wide Web, social, and biological networks, have the scale-free property, i.e., all of their degree distributions follow a power law [1]. Leskovec et al. [28] discover that a wide range of real graphs satisfy the densification and shrinking laws. Other common patterns across networks include small world [57], motif distribution [31], community organization [34], and core-periphery structure [6], validating our hypothesis at the conceptual level.

In the past few years, however, the paradigm of graph learning has been shifted from structural pattern discovery to graph representation learning [11, 14, 25, 39–41, 47, 59], motivated by the recent advances in deep learning [4, 30]. Specifically, graph representation learning converts the vertices, edges, or subgraphs of a graph into low-dimensional embeddings such that vital structural information of the graph is preserved. The learned embeddings from the input graph can be then fed into standard machine learning models for downstream tasks on the same graph.

However, most representation learning work on graphs has thus far focused on learning representations for one single graph or a fixed set of graphs and very limited work can be transferred to out-of-domain data and tasks. Essentially, those representation learning models aim to learn network-specific structural patterns dedicated

*Jie Tang is the corresponding author.
[1]The code is available at https://github.com/THUDM/GCC.

KDD '20, August 23–27, 2020, Virtual Event, CA, USA

ACM ISBN 978-1-4503-7998-4/20/08…$15.00
https://doi.org/10.1145/3394486.3403168

for each dataset. For example, the DeepWalk embedding model [39] learned on the Facebook social graph cannot be applied to other graphs. In view of (1) this limitation of graph representation learning and (2) the prior arts on common structural pattern discovery, a natural question arises here: *can we universally learn transferable representative graph embeddings from networks?*

The similar question has also been asked and pursued in natural language processing [10], computer vision [17], and other domains. To date, the most powerful solution is to pre-train a representation learning model from a large dataset, commonly, under the self-supervised setting. The idea of pre-training is to use the pre-trained model as a good initialization for fine-tuning over (different) tasks on unseen datasets. For example, BERT [10] designs language model pre-training tasks to learn a Transformer encoder [52] from a large corpus. The pre-trained Transformer encoder is then adapted to various NLP tasks [55] by fine-tuning.

Presented work. Inspired by this and the existence of universal graph structural patterns, we study the potential of pre-training representation learning models, specifically, graph neural networks (GNNs), for graphs. Ideally, given a (diverse) set of input graphs, such as the Facebook social graph and the DBLP co-author graph, we aim to pre-train a GNN on them with a self-supervised task, and then fine-tune it on different graphs for different graph learning tasks, such as node classification on the US-Airport graph. The critical question for GNN pre-training here is: *how to design the pre-training task such that the universal structural patterns in and across networks can be captured and further transferred?*

In this work, we present the Graph Contrastive Coding (GCC) framework to learn structural representations across graphs. Conceptually, we leverage the idea of contrastive learning [58] to design the graph pre-training task as instance discrimination. Its basic idea is to sample instances from input graphs, treat each of them as a distinct class of its own, and learn to encode and discriminate between these instances. Specifically, there are three questions to answer for GCC such that it can learn the transferable structural patterns: *(1) what are the instances? (2) what are the discrimination rules? and (3) how to encode the instances?*

In GCC, we design the pre-training task as *subgraph instance discrimination*. Its goal is to distinguish vertices according to their local structures (Cf. Figure 1). For each vertex, we sample subgraphs from its multi-hop ego network as instances. GCC aims to distinguish between subgraphs sampled from a certain vertex and subgraphs sampled from other vertices. Finally, for each subgraph, we use a graph neural network (specifically, the GIN model [59]) as the graph encoder to map the underlying structural patterns to latent representations. As GCC does not assume vertices and subgraphs come from the same graph, the graph encoder is forced to capture universal patterns across different input graphs. Given the pre-trained GCC model, we apply it to unseen graphs for addressing downstream tasks.

To the best of our knowledge, very limited work exists in the field of structural graph representation pre-training to date. A very recent one is to design strategies for pre-training GNNs on labeled graphs with node attributes for specific domains (molecular graphs) [19]. Another recent work is InfoGraph [46], which focuses on learning domain-specific graph-level representations, especially

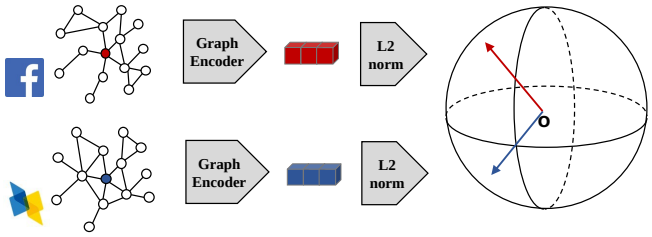


Figure 1: An illustrative example of GCC. In this example, GCC aims to measure the structural similarity between a user from the Facebook social graph and a scholar from the DBLP co-authorship network.

for graph classification tasks. The third related work is by Hu et al. [20], who define several graph learning tasks, such as predicting centrality scores, to pre-train a GCN [25] model on synthetic graphs.

We conduct extensive experiments to demonstrate the performance and transferability of GCC. We pre-train the GCC model on a collection of diverse types of graphs and apply the pre-trained model to three downstream graph learning tasks on ten new graph datasets. The results suggest that the GCC model achieves competitive or better results to the state-of-the-art task-specific graph representation learning models that are trained from scratch. For example, *for node classification on the US-Airport network, GCC pre-trained on the Facebook, IMDB, and DBLP graphs outperforms GraphWave [12], ProNE [64] and Struc2vec [43] which are trained directly on the US-Airport graph, empirically demonstrating our hypothesis at the beginning.*

To summarize, our work makes the following four contributions:

- We formalize the problem of graph neural network pre-training across multiple graphs and identify its design challenges.
- We design the pre-training task as subgraph instance discrimination to capture the universal and transferable structural patterns from multiple input graphs.
- We present the Graph Contrastive Coding (GCC) framework to learn structural graph representations, which leverages contrastive learning to guide the pre-training.
- We conduct extensive experiments to demonstrate that for out-of-domain tasks, GCC can offer comparable or superior performance over dedicated graph-specific models.

2 RELATED WORK

In this section, we review related work of vertex similarity, contrastive learning and graph pre-training.

2.1 Vertex Similarity

Quantifying similarity of vertices in networks/graphs has been extensively studied in the past years. The goal of vertex similarity is to answer questions [26] like "How similar are these two vertices?" or "Which other vertices are most similar to these vertices?" The definition of similarity can be different in different situations. We briefly review the following three types of vertex similarity.

Neighborhood similarity. The basic assumption of neighborhood similarity, a.k.a., proximity, is that vertices closely connected should be considered similar. Early neighborhood similarity measures include Jaccard similarity (counting common neighbors), RWR similarity [36] and SimRank [21], etc. Most recently developed network embedding algorithms, such as LINE [47], DeepWalk [39], node2vec [14], also follow the neighborhood similarity assumption.

Structural similarity. Different from neighborhood similarity which measures similarity by connectivity, structural similarity doesn't even assume vertices are connected. The basic assumption of structural similarity is that vertices with similar local structures should be considered similar. There are two lines of research about modeling structural similarity. The first line defines representative patterns based on domain knowledge. Examples include vertex degree, structural diversity [51], structural hole [7], k-core [2], motif [5, 32], etc. Consequently, models of this genre, such as Struc2vec [43] and RolX [18], usually involve explicit featurization. The second line of research leverages the spectral graph theory to model structural similarity. A recent example is GraphWave [12]. In this work, we focus on structural similarity. Unlike the above two genres, we adopt contrastive learning and graph neural networks to learn structural similarity from data.

Attribute similarity. Real world graph data always come with rich attributes, such as text in citation networks, demographic information in social networks, and chemical features in molecular graphs. Recent graph neural networks models, such as GCN [25], GAT [53], GraphSAGE [16, 62] and MPNN [13], leverage additional attributes as side information or supervised signals to learn representations which are further used to measure vertex similarity.

2.2 Contrastive Learning

Contrastive learning is a natural choice to capture similarity from data. In natural language processing, Word2vec [30] model uses co-occurring words and negative sampling to learn word embeddings. In computer vision, a large collection of work [15, 17, 49, 58] learns self-supervised image representation by minimizing the distance between two views of the same image. In this work, we adopt the InfoNCE loss from Oord et al. [35] and instance discrimination task from Wu et al. [58], as discussed in Section 3.

2.3 Graph Pre-Training

Skip-gram based model. Early attempts to pre-train graph representations are skip-gram based network embedding models inspired by Word2vec [30], such as LINE [47], DeepWalk [39], node2vec [14], and metapath2vec [11]. Most of them follow the neighborhood similarity assumption, as discussed in section 2.1. The representations learned by the above methods are tied up with graphs used to train the models, and can not handle out-of-sample problems. Our Graph Contrastive Coding (GCC) differs from these methods in two aspects. First, GCC focuses on structural similarity, which is orthogonal to neighborhood similarity. Second, GCC can be transferred across graphs, even to graphs never seen during pre-training.

Pre-training graph neural networks. There are several recent efforts to bring ideas from language pre-training [10] to pre-training graph neural networks (GNN). For example, Hu et al. [19] pre-train GNN on labeled graphs, especially molecular graphs, where each vertex (atom) has an atom type (such as C, N, O), and each edge (chemical bond) has a bond type (such as the single bond and double bond). The pre-training task is to recover atom types and chemical bond types in masked molecular graphs. Another related work is by Hu et al. [20], which defines several graph learning tasks to pre-train a GCN [25]. Our GCC framework differs from the above methods in two aspects. First, GCC is for general unlabeled graphs, especially social and information networks. Second, GCC does not involve explicit featurization and pre-defined graph learning tasks.

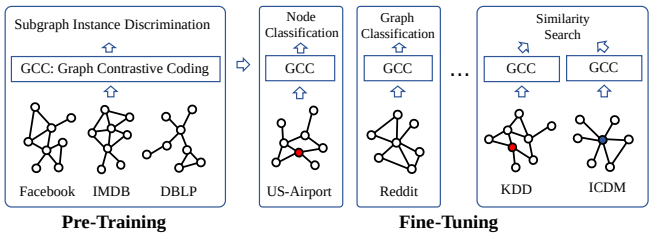


Figure 2: Overall pre-training and fine-tuning procedures for Graph Contrastive Coding (GCC).

3 GRAPH CONTRASTIVE CODING (GCC)

In this section, we formalize the graph neural network (GNN) pre-training problem. To address it, we present the Graph Contrastive Coding (GCC) framework. Figure 2 presents the overview of GCC's pre-training and fine-tuning stages.

3.1 The GNN Pre-Training Problem

Conceptually, given a collection of graphs from various domains, we aim to pre-train a GNN model to capture structural patterns across these graphs in a self-supervised manner. The model should be able to benefit downstream tasks on different datasets. The underlying assumption is that there exist common and transferable structural patterns, such as motifs, across different graphs, as evident in network science literature [28, 32]. One illustrative scenario is that we pre-train a GNN model on Facebook, IMDB, and DBLP graphs with self-supervision, and apply it on the US-Airport network for node classification, as shown in Figure 2.

Formally, the GNN pre-training problem is to learn a function f that maps a vertex to a low-dimensional feature vector, such that f has the following two properties:

- First, *structural similarity*, it maps vertices with similar local network topologies close to each other in the vector space;
- Second, *transferability*, it is compatible with vertices and graphs unseen during pre-training.

As such, the embedding function f can be adopted in various graph learning tasks, such as social role prediction, node classification, and graph classification.

Note that the focus of this work is on structural representation learning without node attributes and node labels, making it different from the common problem setting in graph neural network research. In addition, the goal is to pre-train a structural representation model and apply it to unseen graphs, differing from traditional network

embeddings [14, 39–41, 47] and recent attempts on pre-training graph neural networks with attributed graphs as input and applying them within a specific domain [19].

3.2 GCC Pre-Training

Given a set of graphs, our goal is to pre-train a universal graph neural network encoder to capture the structural patterns behind these graphs. To achieve this, we need to design proper self-supervised tasks and learning objectives for graph structured data.

Inspired by the recent success of contrastive learning in CV [17, 58] and NLP [9, 30], we propose to use *subgraph instance discrimination* as our pre-training task and InfoNCE [35] as our learning objective. The pre-training task treats each subgraph instance as a distinct class of its own and learns to discriminate between these instances. The promise is that it can output representations that captures the similarities between these subgraph instances [17, 58].

From a dictionary look-up perspective, given an encoded query $\boldsymbol{q}$ and a dictionary of $K+1$ encoded keys $\{\boldsymbol{k}_0, \cdots, \boldsymbol{k}_K\}$, contrastive learning looks up a single key (denoted by $\boldsymbol{k}_+$) that $\boldsymbol{q}$ matches in the dictionary. In this work, we adopt InfoNCE such that:

$$\mathcal{L} = -\log \frac{\exp\left(\boldsymbol{q}^\top \boldsymbol{k}_+ / \tau\right)}{\sum_{i=0}^{K} \exp\left(\boldsymbol{q}^\top \boldsymbol{k}_i / \tau\right)} \tag{1}$$

where τ is the temperature hyper-parameter. f_q and f_k are two graph neural networks that encode the query instance x^q and each key instance x^k to d-dimensional representations, denoted by $\boldsymbol{q} = f_q(x^q)$ and $\boldsymbol{k} = f_k(x^k)$.

To instantiate each component in GCC, we need to answer the following three questions:

- **Q1**: How to define subgraph instances in graphs?
- **Q2**: How to define (dis) similar instance pairs in and across graphs, i.e., for a query x^q, which key x^k is the matched one?
- **Q3**: What are the proper graph encoders f_q and f_k?

It is worth noting that in our problem setting, x^q and x^k's are not assumed to be from the same graph. Next we present the design strategies for the GCC pre-training framework by correspondingly answering the aforementioned questions.

Q1: Design (subgraph) instances in graphs. The success of contrastive learning framework largely relies on the definition of the data instance. It is straightforward for CV and NLP tasks to define an instance as an image or a sentence. However, such ideas cannot be directly extended to graph data, as instances in graphs are not clearly defined. Moreover, our pre-training focus is purely on structural representations without additional input features/attributes. This leaves the natural choice of a single vertex as an instance infeasible, as it is not applicable to discriminate between two vertices.

To address this issue, we instead propose to use subgraphs as contrastive instances by extending each single vertex to its local structure. Specifically, for a certain vertex v, we define an instance to be its r-ego network:

Definition 3.1. **A r-ego network.** Let $G = (V, E)$ be a graph, where V denotes the set of vertices and $E \subseteq V \times V$ denotes the set of edges[2]. For a vertex v, its r-neighbors are defined as $S_v = \{u : d(u, v) \leq r\}$ where $d(u, v)$ is the shortest path distance between u and v in the graph G. The r-ego network of vertex v, denoted by G_v, is the sub-graph induced by S_v.

The left panel of Figure 3 shows two examples of 2-ego networks. GCC treats each r-ego network as a distinct class of its own and encourages the model to distinguish similar instances from dissimilar instances. Next, we introduce how to define (dis)similar instances.

Q2: Define (dis)similar instances. In computer vision [17], two random data augmentations (e.g., random crop, random resize, random color jitering, random flip, etc) of the same image are treated as a similar instance pair. In GCC, we consider two random data augmentations of the same r-ego network as a similar instance pair and define the data augmentation as *graph sampling* [27]. Graph sampling is a technique to derive representative subgraph samples from the original graph. Suppose we would like to augment vertex v's r-ego network (G_v), the graph sampling for GCC follows the three steps—random walks with restart (RWR) [50], subgraph induction, and anonymization [22, 29].

(1) **Random walk with restart.** We start a random walk on G from the ego vertex v. The walk iteratively travels to its neighborhood with the probability proportional to the edge weight. In addition, at each step, with a positive probability the walk returns back to the starting vertex v.

(2) **Subgraph induction.** The random walk with restart collects a subset of vertices surrounding v, denoted by $\widetilde{S}_v$. The sub-graph $\widetilde{G}_v$ induced by $\widetilde{S}_v$ is then regarded as an augmented version of the r-ego network G_v. This step is also known as the Induced Subgraph Random Walk Sampling (ISRW).

(3) **Anonymization.** We anonymize the sampled graph $\widetilde{G}_v$ by relabeling its vertices to be $\{1, 2, \cdots, |\widetilde{S}_v|\}$, in arbitrary order[3].

We repeat the aforementioned procedure twice to create two data augmentations, which form a similar instance pair (x^q, x^{k_+}). If two subgraphs are augmented from different r-ego networks, we treat them as a dissimilar instance pair (x^q, x^k) with $k \neq k_+$. It is worth noting that all the above graph operations—random walk with restart, subgraph induction, and anonymization—are available in the DGL package [56].

Discussion on graph sampling. In random walk with restart sampling, the restart probability controls the radius of ego-network (i.e., r) which GCC conducts data augmentation on. In this work, we follow Qiu et al. [42] to use 0.8 as the restart probability. The proposed GCC framework is flexible to other graph sampling algorithms, such as neighborhood sampling [16] and forest fire [27].

Discussion on anonymization. Now we discuss the intuition behind the anonymization step in the above procedure. This step is designed to keep the underlying structural patterns and hide the exact vertex indices. This design avoids learning a trivial solution to subgraph instance discrimination, i.e., simply checking whether vertex indices of two subgraphs match. Moreover, it facilitates the

[2] In this work, we consider undirected edges.
[3] Vertex order doesn't matter because most of graph neural networks are invariant to permutations of their inputs [4].

transfer of the learned model across different graphs as such a model is not associated with a particular vertex set.

Q3: Define graph encoders. Given two sampled subgraphs x^q and x^k, GCC encodes them via two graph neural network encoders f_q and f_k, respectively. Technically, any graph neural networks [4] can be used here as the encoder, and the GCC model is not sensitive to different choices. In practice, we adopt the Graph Isomorphism Network (GIN) [59], a state-of-the-art graph neural network model, as our graph encoder. Recall that we focus on structural representation pre-training while most GNN models require vertex features/attributes as input. To bridge the gap, we propose to leverage the graph structure of each sampled subgraph to initialize vertex features. Specifically, we define the *generalized positional embedding* as follows:

Definition 3.2. **Generalized positional embedding.** For each subgraph, its generalized positional embedding is defined to be the top eigenvectors of its normalized graph Laplacian. Formally, suppose one subgraph has adjacency matrix $\boldsymbol{A}$ and degree matrix $\boldsymbol{D}$, we conduct eigen-decomposition on its normalized graph Laplacian s.t. $\boldsymbol{I}-\boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2} = \boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^\top$, where the top eigenvectors in $\boldsymbol{U}$ [54] are defined as generalized positional embedding.

The generalized positional embedding is inspired by the Transformer model in NLP [52], where the sine and cosine functions of different frequencies are used to define the positional embeddings in word sequences. Such a definition is deeply connected with graph Laplacian as follows.

FACT. *The Laplacian of path graph has eigenvectors:* $\boldsymbol{u}_k(i) = \cos(\pi ki/n - \pi k/2n)$, *for* $1 \le k \le n, 1 \le i \le n$. *Here* n *is the number of vertices in the path graph, and* $\boldsymbol{u}_k(i)$ *is the entry at* i*-th row and* k*-the column of* $\boldsymbol{U}$, *i.e.,* $\boldsymbol{U} = \begin{bmatrix}\boldsymbol{u}_1 & \cdots & \boldsymbol{u}_n\end{bmatrix}$.

The above fact shows that the positional embedding in sequence models can be viewed as Laplacian eigenvectors of path graphs. This inspires us to generalize the positional embedding from path graphs to arbitrary graphs. The reason for using the normalized graph Laplacian rather than the unnormalized version is that path graph is a regular graph (i.e., with constant degrees) while real-world graphs are often irregular and have skewed degree distributions. In addition to the generalized positional embedding, we also add the one-hot encoding of vertex degrees [59] and the binary indicator of the ego vertex [42] as vertex features. After encoded by the graph encoder, the final d-dimensional output vectors are then normalized by their L2-Norm [17].

A running example. We illustrate a running example of GCC pre-training in Figure 3. For simplicity, we set the dictionary size to be 3. GCC first randomly augments two subgraphs x^q and x^{k_0} from a 2-ego network on the left panel of Figure 3. Meanwhile, another two subgraphs, x^{k_1} and x^{k_2}, are generated from a noise distribution — in this example, they are randomly augmented from another 2-ego network on the left panel of Figure 3. Then the two graph encoders, f_q and f_k, map the query and the three keys to low-dimensional vectors — $\boldsymbol{q}$ and $\{\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2\}$. Finally, the contrastive loss in Eq. 1 encourages the model to recognize (x^q, x^{k_0}) as a similar instance pair and distinguish them from dissimilar instances, i.e., $\{x^{k_1}, x^{k_2}\}$.

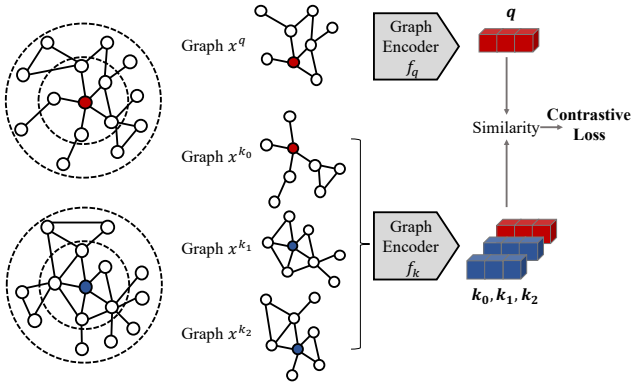


Figure 3: A running example of GCC pre-training.

Learning. In contrastive learning, it is required to maintain the K-size dictionary and encoders. Ideally, in Eq. 1, the dictionary should cover as many instances as possible, making K extremely large. However, due to the computational constraints, we usually design and adopt economical strategies to effectively build and maintain the dictionary, such as end-to-end (E2E) and momentum contrast (MoCo) [17]. We discuss the two strategies as follows.

E2E samples mini-batches of instances and considers samples in the same mini-batch as the dictionary. The objective in Eq. 1 is then optimized with respect to parameters of both f_q and f_k, both of which can accept gradient updates by backpropagation consistently. The main drawback of E2E is that the dictionary size is constrained by the batch size.

MoCo is designed to increase the dictionary size without additional backpropagation costs. Concretely, MoCo maintains a queue of samples from preceding mini-batches. During optimization, MoCo only updates the parameters of f_q (denoted by θ_q) by backpropagation. The parameters of f_k (denoted by θ_k) are not updated by gradient descent. He et al. [17] propose a momentum-based update rule for θ_k. Formally, MoCo updates θ_k by $\theta_k \leftarrow m\theta_k + (1-m)\theta_q$, where $m \in [0, 1)$ is a momentum hyper-parameter. The above momentum update rule gradually propagates the update in θ_q to θ_k, making θ_k evolve smoothly and consistently. In summary, MoCo achieves a larger dictionary size at the expense of dictionary consistency, i.e., the key representations in the dictionary are encoded by a smoothly-varying key encoder.

In addition to E2E and MoCo, there are other contrastive learning mechanisms to maintain the dictionary, such as memory bank [58]. Recently, He et al. [17] show that MoCo is a more effective option than memory bank in computer vision tasks. Therefore, we mainly focus on E2E and MoCo for GCC.

3.3 GCC Fine-Tuning

Downstream tasks. Downstream tasks in graph learning generally fall into two categories—graph-level and node-level, where the target is to predict labels of graphs or nodes, respectively. For graph-level tasks, the input graph itself can be encoded by GCC to achieve the representation. For node-level tasks, the node representation can be defined by encoding its r-ego networks (or subgraphs augmented from its r-ego network). In either case, the encoded

representations are then fed into downstream tasks to predict task-specific outputs.

Freezing vs. full fine-tuning. GCC offers two fine-tuning strategies for downstream tasks—the freezing mode and full fine-tuning mode. In the freezing mode, we freeze the parameters of the pre-trained graph encoder f_q and treat it as a static feature extractor, then the classifiers catering for specific downstream tasks are trained on top of the extracted features. In the full fine-tuning mode, the graph encoder f_q initialized with pre-trained parameters is trained end-to-end together with the classifier on a downstream task. More implementation details about fine-tuning are available in Section 4.2.

GCC as a local algorithm. As a graph algorithm, GCC belongs to the local algorithm category [45, 48], in which the algorithms only involve local explorations of the input (large-scale) network, since GCC explores local structures by random walk based graph sampling methods. Such a property enables GCC to scale to large-scale graph learning tasks and to be friendly to the distributed computing setting.

4 EXPERIMENTS

In this section, we evaluate GCC on three graph learning tasks—node classification, graph classification, and similarity search, which have been commonly used to benchmark graph learning algorithms [12, 43, 46, 59, 60]. We first introduce the self-supervised pre-training settings in Section 4.1, and then report GCC fine-tuning results on those three graph learning tasks in Section 4.2.

4.1 Pre-Training

Datasets. Our self-supervised pre-training is performed on six graph datasets, which can be categorized into two groups—*academic graphs* and *social graphs*. As for academic graphs, we collect the Academia dataset from NetRep [44] as well as two DBLP datasets from SNAP [61] and NetRep [44], respectively. As for social graphs, we collect Facebook and IMDB datasets from NetRep [44], as well as a LiveJournal dataset from SNAP [3]. Table 1 presents the detailed statistics of datasets for pre-training.

Pre-training settings. We train for 75,000 steps and use Adam [24] for optimization with learning rate of 0.005, $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1 \times 10^{-8}$, weight decay of 1e-4, learning rate warmup over the first 7,500 steps, and linear decay of the learning rate after 7,500 steps. Gradient norm clipping is applied with range $[-1, 1]$. For MoCo, we use mini-batch size of 32, dictionary size of 16,384, and momentum m of 0.999. For E2E, we use mini-batch size of 1,024. For both MoCo and E2E, the temperature τ is set as 0.07, and we adopt GIN [59] with 5 layers and 64 hidden units each layer as our encoders. Detailed hyper-parameters can be found in Table 6 in the Appendix.

4.2 Downstream Task Evaluation

In this section, we apply GCC to three graph learning tasks including node classification, graph classification, and similarity search. As prerequisites, we discuss the two fine-tuning strategies of GCC as well as the baselines we compare with.

Fine-tuning. As we discussed in Section 3.3, we adopt two fine-tuning strategies for GCC. We select logistic regression or SVM from the scikit-learn [38] package as the linear classifier for the freezing strategy[4]. As for the full fine-tuning strategy, we use the Adam optimizer with learning rate 0.005, learning rate warmup over the first 3 epochs, and linear learning rate decay after 3 epochs.

Baselines. Baselines can be categorized into two categories. In the first category, the baseline models learn vertex/graph representations from unlabeled graph data and then feed them into logistic regression or SVM. Examples include DGK [60], Struc2vec [43], GraphWave [12], graph2vec[33] and InfoGraph [46]. GCC with the freezing setting belongs to this category. In the second category, the models are optimized in an end-to-end supervised manner. Examples include DGCNN [66] and GIN [59]. GCC with the full fine-tuning setting belongs to this category.

For a fair comparison, we fix the representation dimension of all models to be 64 except graph2vec and InfoGraph[5]. The details of baselines will be discussed later.

4.2.1 *Node Classification.*

Setup. The node classification task is to predict unknown node labels in a partially labeled network. To evaluate GCC, we sample a subgraph centered at each vertex and apply GCC on it. Then the obtained representation is fed into an output layer to predict the node label. As for datasets, we adopt US-Airport [43] and H-index [63]. US-Airport consists of the airline activity data among 1,190 airports. The 4 classes indicate different activity levels of the airports. H-index is a co-authorship graph extracted from OAG [63]. The labels indicate whether the h-index of the author is above or below the median.

Experimental results. We compare GCC with ProNE [64], GraphWave [12], and Struc2vec [43]. Table 2 represents the results. It is worth noting that, under the freezing setting, the graph encoder in GCC is not trained on either US-Airport or H-Index dataset, which other baselines use as training data. This places GCC at a disadvantage. However, GCC (MoCo, freeze) performs competitively to Struc2vec in US-Airport, and achieves the best performance in H-index where Struc2vec cannot finish in one day. Moreover, GCC can be further boosted by fully fine-tuning on the target US-Airport or H-Index domain.

4.2.2 *Graph Classification.*

Setup. We use five datasets from Yanardag and Vishwanathan [60]—COLLAB, IMDB-BINARY, IMDB-MULTI, REDDITBINARY and REDDIT-MULTI5K, which are widely benchmarked in recent graph classification models [19, 46, 66]. Each dataset is a set of graphs where each graph is associated with a label. To evaluate GCC on this task, we use raw input graphs as the input of GCC. Then the encoded graph-level representation is fed into a classification

[4] In node classification tasks, we follow Struc2vec to use logistic regression. For graph classification tasks, we follow DGK [60] and GIN [59] to use SVM [8].
[5] We allow them to use their preferred dimension size in their papers: graph2vec uses 1024 and InfoGraph uses 512.

Table 1: Datasets for pre-training, sorted by number of vertices.

Dataset	Academia	DBLP (SNAP)	DBLP (NetRep)	IMDB	Facebook	LiveJournal
$\|V\|$	137,969	317,080	540,486	896,305	3,097,165	4,843,953
$\|E\|$	739,384	2,099,732	30,491,458	7,564,894	47,334,788	85,691,368

Table 2: Node classification.

Datasets	US-Airport	H-index
$\|V\|$	1,190	5,000
$\|E\|$	13,599	44,020
ProNE	62.3	69.1
GraphWave	60.2	70.3
Struc2vec	**66.2**	> 1 Day
GCC (E2E, freeze)	64.8	**78.3**
GCC (MoCo, freeze)	65.6	75.2
GCC (rand, full)	64.2	76.9
GCC (E2E, full)	**68.3**	80.5
GCC (MoCo, full)	67.2	**80.6**

Table 3: Graph classification.

Datasets	IMDB-B	IMDB-M	COLLAB	RDT-B	RDT-M
# graphs	1,000	1,500	5,000	2,000	5,000
# classes	2	3	3	2	5
Avg. # nodes	19.8	13.0	74.5	429.6	508.5
DGK	67.0	44.6	73.1	78.0	41.3
graph2vec	71.1	50.4	–	75.8	47.9
InfoGraph	**73.0**	**49.7**	–	82.5	53.5
GCC (E2E, freeze)	71.7	49.3	74.7	87.5	52.6
GCC (MoCo, freeze)	72.0	49.4	**78.9**	**89.8**	**53.7**
DGCNN	70.0	47.8	73.7	–	–
GIN	**75.6**	**51.5**	80.2	**89.4**	**54.5**
GCC (rand, full)	**75.6**	50.9	79.4	87.8	52.1
GCC (E2E, full)	70.8	48.5	79.0	86.4	47.4
GCC (MoCo, full)	73.8	50.3	**81.1**	87.6	53.0

layer to predict the label of the graph. We compare GCC with several recent developed graph classification models, including Deep Graph Kernel (DGK) [60], graph2vec [33], InfoGraph [46], DGCNN [66] and GIN [59]. Among these baselines, DGK, graph2vec and InfoGraph belong to the first category, while DGCNN and GIN belong to the second category.

Experimental results. Table 3 shows the comparison. In the first category, GCC (MoCo, freeze) performs competitively to InfoGraph in IMDB-B and IMDB-M, while achieves the best performance in other datasets. Again, we want to emphasize that DGK, graph2vec and InfoGraph all need to be pre-trained on target domain graphs, but GCC only relies on the graphs listed in Table 1 for pre-training. In the second category, we compare GCC with DGCNN and GIN. GCC achieves better performance than DGCNN and comparable performance to GIN. GIN is a recently proposed SOTA model for graph classification. We follow the instructions in the paper [59] to train GIN and report the detailed results in Table 7 in the Appendix. We can see that, in each dataset, the best performance of GIN is achieved by different hyper-parameters. And by varying hyper-parameters, GIN's performance could be sensitive. However, GCC on all datasets shares the same pre-training/fine-tuning hyper-parameters, showing its robustness on graph classification.

Table 4: Top-k similarity search ($k = 20, 40$).

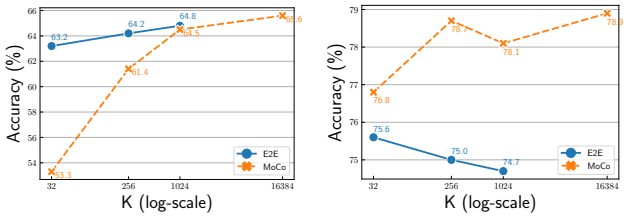
	KDD-ICDM		SIGIR-CIKM		SIGMOD-ICDE	
$\|V\|$	2,867	2,607	2,851	3,548	2,616	2,559
$\|E\|$	7,637	4,774	6,354	7,076	8,304	6,668
# groud truth		697		874		898
k	20	40	20	40	20	40
Random	0.0198	0.0566	0.0223	0.0447	0.0221	0.0521
RolX	0.0779	0.1288	0.0548	0.0984	0.0776	0.1309
Panther++	0.0892	0.1558	**0.0782**	0.1185	0.0921	0.1320
GraphWave	0.0846	**0.1693**	0.0549	0.0995	**0.0947**	**0.1470**
GCC (E2E)	**0.1047**	0.1564	0.0549	**0.1247**	0.0835	0.1336
GCC (MoCo)	0.0904	0.1521	0.0652	0.1178	0.0846	0.1425

4.2.3 *Top-k Similarity Search.*

Setup. We adopt the co-author dataset from Zhang et al. [65], which are the conference co-author graphs of KDD, ICDM, SIGIR, CIKM, SIGMOD, and ICDE. The problem of top-k similarity search is defined as follows. Given two graphs G_1 and G_2, for example KDD and ICDM co-author graphs, we want to find the most similar vertex v from G_1 for each vertex u in G_2. In this dataset, the ground truth is defined to be authors publish in both conferences. Note that similarity search is an unsupervised task, so we evaluate GCC without fine-tuning. Especially, we first extract two subgraphs centered at u and v by random walk with restart graph sampling. After encoding them by GCC, we measure the similarity score between u and v to be the inner product of their representations. Finally, by sorting the above scores, we use HITS@10 (top-10 accuracy) to measure the performance of different methods. We compare GCC with RolX [18], Panther++ [65] and GraphWave [12]. We also provide random guess results for reference.

Experimental results. Table 4 presents the performance of different methods on top-k similarity search task in three co-author networks. We can see that, compared with Panther++ [65] and GraphWave [12] which are trained in place on co-author graphs, simply applying pre-trained GCC can be competitive.

Overall, we show that a graph neural network encoder pre-trained on several popular graph datasets can be directly adapted to new graph datasets and unseen graph learning tasks. More importantly, compared with models trained from scratch, the reused model achieves competitive and sometimes better performance. This demonstrates the transferability of graph structural patterns and the effectiveness of our GCC framework in capturing these patterns.



(a) US-Airport **(b) COLLAB**

Figure 4: Comparison of contrastive loss mechanisms.

4.3 Ablation Studies

Effect of pre-training. It is still not clear if GCC's good performance is due to pre-training or the expression power of its GIN [59] encoder. To answer this question, we fully fine-tune GCC with its GIN encoder randomly initialized, which is equivalent to train a GIN encoder from scratch. We name this model GCC (rand), as shown in Table 2 and Table 3. In all datasets except IMDB-B, GCC (MoCo) outperforms its randomly initialized counterpart, showing that pre-training always provides a better start point for fine-tuning than random initialization. For IMDB-B, we attribute it to the domain shift between pre-training data and down-stream tasks.

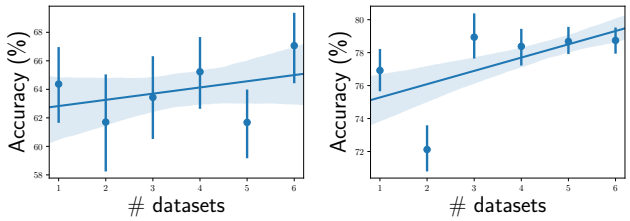
Contrastive loss mechanisms. The common belief is that MoCo has stronger expression power than E2E [17], and a larger dictionary size K always helps. We also observe such trends, as shown in Figure 4. However, the effect of a large dictionary size is not as significant as reported in computer vision tasks [17]. For example, MoCo ($K = 16384$) merely outperforms MoCo ($K = 1024$) by small margins in terms of accuracy — 1.0 absolute gain in US-Airport and 0.8 absolute gain in COLLAB. However, training MoCo is much more economical than training E2E. E2E ($K = 1024$) takes 5 days and 16 hours, while MoCo ($K = 16384$) only needs 9 hours. Detailed training time can be found in Table 6 in the Appendix.

Table 5: Momentum ablation.

momentum m	0	0.9	0.99	0.999	0.9999
US-Airport	62.3	63.2	63.7	**65.6**	61.5
COLLAB	76.6	75.1	77.4	78.9	**79.4**

Momentum. As mentioned in MoCo [17], momentum m plays a subtle role in learning high-quality representations. Table 5 shows accuracy with different momentum values on US-Airport and COLLAB datasets. For US-Airport, the best performance is reached by $m = 0.999$, which is the desired value in [17], showing that building a consistent dictionary is important for MoCo. However, in COLLAB, it seems that a larger momentum value brings better performance. Moreover, we do not observe the "training loss oscillation" reported in [17] when setting $m = 0$. GCC (MoCo) converges well, but the accuracy is much worse.

Pre-training datasets. We ablate the number of datasets used for pre-training. To avoid enumerating a combinatorial space, we pre-train with first several datasets in Table 1, and report the 10-fold validation accuracy scores on US-Airport and COLLAB, respectively. For example, when using one dataset for pre-training, we select Academia; when using two, we choose Academia and DBLP (SNAP); and so on. We present ordinary least squares (OLS) estimates of the relationship between the number of datasets and the model performance. As shown in Figure 5, we can observe a trend towards higher accuracy when using more datasets for pre-training. On average, adding one more dataset leads to 0.43 and 0.81 accuracy (%) gain on US-Airport and COLLAB, respectively[6].



(a) US-Airport: $y = 0.4344x + 62.394$ **(b) COLLAB:** $y = 0.8065x + 74.4737$

Figure 5: Ablation study on pre-training datasets.

5 CONCLUSION

In this work, we study the pre-training of graph neural networks with the goal of characterizing and transferring structural representations in social and information networks. We present Graph Contrastive Coding (GCC), which is a graph-based contrastive learning framework to pre-train graph neural networks from multiple graph datasets. The pre-trained graph neural network achieves competitive performance to its supervised trained-from-scratch counterparts in three graph learning tasks on ten graph datasets. In the future, we plan to benchmark more graph learning tasks on more diverse graph datasets, such as the protein-protein association networks.

Acknowledgements. The work is supported by the National Key R&D Program of China (2018YFB1402600), NSFC for Distinguished Young Scholar (61825602), and NSFC (61836013).

REFERENCES

[1] Réka Albert and Albert-László Barabási. 2002. Statistical mechanics of complex networks. *Reviews of modern physics* 74, 1 (2002), 47.
[2] J Ignacio Alvarez-Hamelin, Luca Dall'Asta, Alain Barrat, and Alessandro Vespignani. 2006. Large scale networks fingerprinting and visualization using the k-core decomposition. In *Advances in neural information processing systems*. 41–50.
[3] Lars Backstrom, Dan Huttenlocher, Jon Kleinberg, and Xiangyang Lan. 2006. Group formation in large social networks: membership, growth, and evolution. In *KDD '06*. 44–54.
[4] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018).
[5] Austin R Benson, David F Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.
[6] Stephen P Borgatti and Martin G Everett. 2000. Models of core/periphery structures. *Social networks* 21, 4 (2000), 375–395.
[7] Ronald S Burt. 2009. *Structural holes: The social structure of competition*. Harvard university press.
[8] Chih-Chung Chang and Chih-Jen Lin. 2011. LIBSVM: A library for support vector machines. *ACM transactions on intelligent systems and technology (TIST)* 2, 3 (2011), 1–27.

[6]The effect on US-Airport is positive, but statistically insignificant (p-value $= 0.231$), while the effect on COLLAB is positive and significant (p-value $\ll 0.001$).

[9] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. 2019. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. In *ICLR '19*.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT '19*. 4171–4186.

[11] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. 2017. metapath2vec: Scalable representation learning for heterogeneous networks. In *KDD '17*. 135–144.

[12] Claire Donnat, Marinka Zitnik, David Hallac, and Jure Leskovec. 2018. Learning structural node embeddings via diffusion wavelets. In *KDD '18*. 1320–1329.

[13] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *ICML '17*. JMLR. org, 1263–1272.

[14] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *KDD '16*. 855–864.

[15] Raia Hadsell, Sumit Chopra, and Yann LeCun. 2006. Dimensionality reduction by learning an invariant mapping. In *CVPR '06*, Vol. 2. IEEE, 1735–1742.

[16] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in neural information processing systems*. 1024–1034.

[17] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *CVPR '20*. 9729–9738.

[18] Keith Henderson, Brian Gallagher, Tina Eliassi-Rad, Hanghang Tong, Sugato Basu, Leman Akoglu, Danai Koutra, Christos Faloutsos, and Lei Li. 2012. Rolx: structural role extraction & mining in large graphs. In *KDD '12*. 1231–1239.

[19] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2019. Pre-training graph neural networks. In *ICLR '19*.

[20] Ziniu Hu, Changjun Fan, Ting Chen, Kai-Wei Chang, and Yizhou Sun. 2019. Unsupervised Pre-Training of Graph Convolutional Networks. *ICLR 2019 Workshop: Representation Learning on Graphs and Manifolds* (2019).

[21] Glen Jeh and Jennifer Widom. 2002. SimRank: a measure of structural-context similarity. In *KDD '02*. 538–543.

[22] Yilun Jin, Guojie Song, and Chuan Shi. 2019. GraLSP: Graph Neural Networks with Local Structural Patterns. *arXiv preprint arXiv:1911.07675* (2019).

[23] Kristian Kersting, Nils M. Kriege, Christopher Morris, Petra Mutzel, and Marion Neumann. 2016. Benchmark Data Sets for Graph Kernels. http://graphkernels.cs.tu-dortmund.de

[24] Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. *ICLR '15*.

[25] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR '17*.

[26] Elizabeth A Leicht, Petter Holme, and Mark EJ Newman. 2006. Vertex similarity in networks. *Physical Review E* 73, 2 (2006), 026120.

[27] Jure Leskovec and Christos Faloutsos. 2006. Sampling from large graphs. In *KDD '06*. 631–636.

[28] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. 2005. Graphs over time: densification laws, shrinking diameters and possible explanations. In *KDD '05*. 177–187.

[29] Silvio Micali and Zeyuan Allen Zhu. 2016. Reconstructing markov processes from independent and anonymous experiments. *Discrete Applied Mathematics* 200 (2016), 108–122.

[30] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*. 3111–3119.

[31] Ron Milo, Shalev Itzkovitz, Nadav Kashtan, Reuven Levitt, Shai Shen-Orr, Inbal Ayzenshtat, Michal Sheffer, and Uri Alon. 2004. Superfamilies of evolved and designed networks. *Science* 303, 5663 (2004), 1538–1542.

[32] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. 2002. Network motifs: simple building blocks of complex networks. *Science* 298, 5594 (2002), 824–827.

[33] Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. 2017. graph2vec: Learning distributed representations of graphs. *arXiv preprint arXiv:1707.05005* (2017).

[34] Mark EJ Newman. 2006. Modularity and community structure in networks. *Proceedings of the national academy of sciences* 103, 23 (2006), 8577–8582.

[35] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[36] Jia-Yu Pan, Hyung-Jeong Yang, Christos Faloutsos, and Pinar Duygulu. 2004. Automatic multimedia cross-modal correlation discovery. In *KDD '04*. 653–658.

[37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*. 8024–8035.

[38] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of machine learning research* 12, Oct (2011), 2825–2830.

[39] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *KDD '14*. 701–710.

[40] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Chi Wang, Kuansan Wang, and Jie Tang. 2019. Netsmf: Large-scale network embedding as sparse matrix factorization. In *The World Wide Web Conference*. 1509–1520.

[41] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. 2018. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec. In *WSDM '18*. 459–467.

[42] Jiezhong Qiu, Jian Tang, Hao Ma, Yuxiao Dong, Kuansan Wang, and Jie Tang. 2018. Deepinf: Social influence prediction with deep learning. In *KDD '18*. 2110–2119.

[43] Leonardo FR Ribeiro, Pedro HP Saverese, and Daniel R Figueiredo. 2017. struc2vec: Learning node representations from structural identity. In *KDD '17*. 385–394.

[44] Scott C Ritchie, Stephen Watts, Liam G Fearnley, Kathryn E Holt, Gad Abraham, and Michael Inouye. 2016. A scalable permutation approach reveals replication and preservation patterns of network modules in large datasets. *Cell systems* 3, 1 (2016), 71–82.

[45] Daniel A Spielman and Shang-Hua Teng. 2013. A local clustering algorithm for massive graphs and its application to nearly linear time graph partitioning. *SIAM Journal on computing* 42, 1 (2013), 1–26.

[46] Fan-Yun Sun, Jordan Hoffman, Vikas Verma, and Jian Tang. 2019. InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization. In *ICLR '19*.

[47] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *WWW '15*. 1067–1077.

[48] Shang-Hua Teng et al. 2016. Scalable algorithms for data and network analysis. *Foundations and Trends® in Theoretical Computer Science* 12, 1–2 (2016), 1–274.

[49] Yonglong Tian, Dilip Krishnan, and Phillip Isola. 2019. Contrastive multiview coding. *arXiv preprint arXiv:1906.05849* (2019).

[50] Hanghang Tong, Christos Faloutsos, and Jia-Yu Pan. 2006. Fast random walk with restart and its applications. In *ICDM '06*. IEEE, 613–622.

[51] Johan Ugander, Lars Backstrom, Cameron Marlow, and Jon Kleinberg. 2012. Structural diversity in social contagion. *Proceedings of the National Academy of Sciences* 109, 16 (2012), 5962–5966.

[52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.

[53] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph attention networks. *ICLR '18* (2018).

[54] Ulrike Von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and computing* 17, 4 (2007), 395–416.

[55] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *ICLR '19*.

[56] Minjie Wang, Lingfan Yu, Da Zheng, Quan Gan, Yu Gai, Zihao Ye, Mufei Li, Jinjing Zhou, Qi Huang, Chao Ma, et al. 2019. Deep graph library: Towards efficient and scalable deep learning on graphs. *arXiv preprint arXiv:1909.01315* (2019).

[57] Duncan J Watts and Steven H Strogatz. 1998. Collective dynamics of small-world networks. *nature* 393, 6684 (1998), 440.

[58] Zhirong Wu, Yuanjun Xiong, Stella X Yu, and Dahua Lin. 2018. Unsupervised feature learning via non-parametric instance discrimination. In *CVPR '18*. 3733–3742.

[59] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *ICLR '19*.

[60] Pinar Yanardag and SVN Vishwanathan. 2015. Deep graph kernels. In *KDD '15*. 1365–1374.

[61] Jaewon Yang and Jure Leskovec. 2015. Defining and evaluating network communities based on ground-truth. *Knowledge and Information Systems* 42, 1 (2015), 181–213.

[62] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD '18*. 974–983.

[63] Fanjin Zhang, Xiao Liu, Jie Tang, Yuxiao Dong, Peiran Yao, Jie Zhang, Xiaotao Gu, Yan Wang, Bin Shao, Rui Li, and et al. 2019. OAG: Toward Linking Large-Scale Heterogeneous Entity Graphs. In *KDD '19*. 2585–2595.

[64] Jie Zhang, Yuxiao Dong, Yan Wang, Jie Tang, and Ming Ding. 2019. ProNE: fast and scalable network representation learning. In *IJCAI '19*. 4278–4284.

[65] Jing Zhang, Jie Tang, Cong Ma, Hanghang Tong, Yu Jing, and Juanzi Li. 2015. Panther: Fast top-k similarity search on large networks. In *KDD '15*. 1445–1454.

[66] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An end-to-end deep learning architecture for graph classification. In *AAAI '18*.

A APPENDIX

Table 6: Pre-training hyper-parameters for E2E and MoCo.

Hyper-parameter	E2E	MoCo
Batch size	1024	32
Restart probability	0.8	0.8
Dictionary size K	1023	16384
Temperature τ	0.07	0.07
Momentum m	NA	0.999
Warmup steps	7,500	7,500
Weight decay	1e-5	1e-5
Training steps	75,000	75,000
Initial learning rate	0.005	0.005
Learning rate decay	Linear	Linear
Adam ϵ	1e-8	1e-8
Adam β_1	0.9	0.9
Adam β_2	0.999	0.999
Gradient clipping	1.0	1.0
Number of layers	5	5
Hidden units per layer	64	64
Dropout rate	0.5	0.5
Degree embedding size	16	16

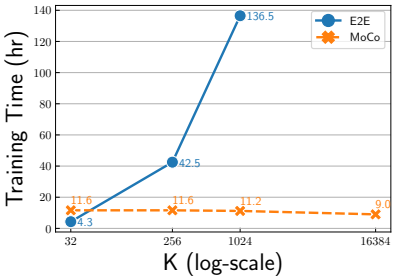


Figure 6: Pre-training time of different contrastive loss mechanisms and dictionary size K.

A.1 Implementation Details

A.1.1 *Hardware Configuration.*

The experiments are conducted on Linux servers equipped with an Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz, 256GB RAM and 8 NVIDIA 2080Ti GPUs.

A.1.2 *Software Configuration.*

All models are implemented in PyTorch [37] version 1.3.1, DGL [56] version 0.4.1 with CUDA version 10.1, scikit-learn version 0.20.3 and Python 3.6. Our code and datasets will be available.

A.1.3 *Pre-training.*

The detailed hyper-parameters are listed in Table 6. Training times of GCC variants are listed in Figure 6.[7] The training time of GCC (E2E) grows sharply with the dictionary size K while GCC (MoCo) roughly remains the same, which indicates that MoCo is more economical and easy to scale with larger dictionary size.

[7]The table shows the elapsed real time for pre-training, which might be affected by other programs running on the server.

Table 7: Performance of GIN model under various hyper-parameter configurations.

batch	dropout	degree	IMDB-B	IMDB-M	COLLAB	RDT-B	RDT-M
32	0	no	72.9	48.7	69.2	83.5	52.2
32	0.5	no	71.6	49.1	66.2	77.5	51.7
128	0	no	**75.6**	50.9	73.0	**89.4**	**54.5**
128	0.5	no	74.6	51.0	72.3	81.8	53.5
32	0	yes	73.9	51.1	79.7	77.0	46.2
32	0.5	yes	74.5	50.1	79.1	77.4	45.6
128	0	yes	73.3	51.0	79.8	76.7	45.6
128	0.5	yes	73.1	**51.5**	**80.2**	77.4	45.8

A.2 Baselines

A.2.1 *Node Classification.*

GraphWave [12]. We download the authors' official source code and keep all the training settings as the same. The implementation requires a networkx graph and time points as input. We convert our dataset to the networkx format, and use automatic selection of the range of scales provided by the authors. We set the output embedding dimension to 64.

Code: https://github.com/snap-stanford/graphwave/.

Struc2vec [43]. We download the authors' official source code and use default hyper-parameters provided by the authors: (1) walk length = 80; (2) number of walks = 10; (3) window size = 10; (4) number of iterations = 5.

The only modifications we do are: (1) number of dimensions = 64; (2) number of workers = 48 to speed up training.

We find the method hard to scale on the H-index datasets although we set the number of workers to 48, compared to 4 by default. We keep the code running for 24 hours on the H-index datasets and it failed to finish. We observed that the sampling strategy in Struc2vec takes up most of the time, as illustrated in the original paper.

Code: https://github.com/leoribeiro/struc2vec.

ProNE [64]. We download the authors' official code and keep hyper-parameters as the same: (1) step = 10; (2) θ = 0.5; (3) μ = 0.2. The dimension size is set to 64.

Code: https://github.com/THUDM/ProNE/.

A.2.2 *Graph Classification.*

DGK [60], graph2vec [33], InfoGraph [46], DGCNN [66]. We adopt the reported results in these papers. Our experimental setting is exactly the same except for the dimension size. Note that graph2vec uses 1024 and InfoGraph uses 512 as the dimension size. Following GIN, we use 64.

GIN [59]. We use the official code released by [59] and follow exactly the procedure described in their paper: the hyper-parameters tuned for each dataset are: (1) the number of hidden units $\in \{16, 32\}$ for bioinformatics graphs and 64 for social graphs; (2) the batch size $\in \{32, 128\}$; (3) the dropout ratio $\in \{0, 0.5\}$ after the dense layer; (4) the number of epochs, i.e., a single epoch with the best

cross-validation accuracy averaged over the 10 folds was selected. We report the obtained results in Table 7.

Code: https://github.com/weihua916/powerful-gnns.

A.2.3 Top-k Similarity Search.

Random, RolX [18], Panther++ [65]. We obtainÂăthe experimental results for these baselines from Zhang et al. [65].

Code: https://github.com/yuikns/panther/.

GraphWave [12]. Embeddings computed by the GraphWave method also have the ability to generalize across graphs. The authors evaluated on synthetic graphs in their paper which are not publicly available. To compare with GraphWave on the co-author datasets, we compute GraphWave embeddings given two graphs G_1 and G_2 and follow the same procedure mentioned in section 4.2.2 to compute the HITS@10 (top-10 accuracy) score.

Code: https://github.com/snap-stanford/graphwave/.

A.3 Datasets

A.3.1 Node Classification Datasets.

US-Airport [8] We obtain the US-Airport dataset directly from Ribeiro et al. [43].

H-index [9] We create the H-index dataset, a co-authorship graph extracted from OAG [63]. Since the original OAG co-authorship graph has millions of nodes, it is too large as a node classification benchmark. Therefore, we implemented the following procedure to extract smaller subgraphs from OAG:

(1) Select an initial vertex set V_s in OAG;

(2) Run breadth first search (BFS) from V_s until N nodes are visited;

(3) Return the sub-graph induced by the visited N nodes.

We set $N = 5,000$, and randomly select 20 nodes from top 200 nodes with largest degree as he initial vertex set in step (1).

A.3.2 Graph Classification Datasets. [10]

We download COLLAB, IMDB-BINARY, IMDB-MULTI, REDDIT-BINARY and REDDIT-MULTI5K from Benchmark Data Sets for Graph Kernels [23].

A.3.3 Top-k Similarity Search Datasets. [11]

We obtain the paired conference co-author datasets, including KDD-ICDM, SIGIR-CIKM, SIGMOD-ICDE, from the Zhang et al. [65] and make them publicly available with the permission of the original authors.

[8] https://github.com/leoribeiro/struc2vec/tree/master/graph.
[9] https://www.openacademic.ai/oag/.
[10] https://ls11-www.cs.tu-dortmund.de/staff/morris/graphkerneldatasets
[11] https://github.com/yuikns/panther